



温州肯恩大学

WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01

Python Programming for Business

HTML Scraping (requests + BeautifulSoup)

Dawei Chen

Week 10-2, 30 April 2026

Where We Are

Date	Topic	Status
4/16	Big Picture & GitHub Literacy	✓ Done
4/21	API-Based Data Collection	✓ Done
4/23	JSON & Semi-Structured Data	✓ Done
4/28	<i>WKU Student Research Day (No Lecture)</i>	
4/30	HTML Scraping (requests + BeautifulSoup)	◀ Today

Assignment 3 due **May 10, Sunday, 23:59**

Instructions are available on Canvas

Learning Objectives

By the end of this session, you should be able to:

- 1 Explain when and why HTML scraping is needed (*vs. API*)
- 2 Understand **HTML structure**: tags, attributes, and content
- 3 Use browser **Developer Tools** to inspect a webpage's structure
- 4 **Run and modify** an AI-generated scraping notebook (`requests` + `BeautifulSoup`)

Recap: The requests Module



Quick check: What is an API?

The Restaurant Analogy

A defined interface for software to request services or data from another system

```
response = requests.get(API_URL, headers=test_headers, params=test_params)
```

Parameters used:

- **url** — where to send the request
- **headers** — authentication (token)
- **params** — query parameters

What it returns:

- **Status code** — HTTP status (e.g., 200 OK, 404 Not Found, 403 Forbidden)
- **text** — raw response body
- **.json()** — parse response body as JSON (APIs)

Recap: The `requests` Module



`requests.get()` always returns whatever the **server** sends back.

The function doesn't decide the format — the server does.

Call an API URL → **API server** returns **JSON** (structured data)

https://api.github.com/search/repositories?q=AI-skills&sort=stars&per_page=10

Call a webpage URL → **Web server** returns **HTML** (a webpage for browsers)

<https://github.com/search?q=AI-skills&type=repositories&s=stars&o=desc>

Same function and HTTP protocol, but different servers return different responses.

When API vs. When Scraping?

API available → Use API

- Returns structured JSON
- Official, documented, reliable
- Rate limits are clear

E.g. GitHub, Reddit, Spotify

No API → Use scraping

- Returns raw HTML — needs parsing
- Unofficial, fragile, can break
- Must be polite (delays, robots.txt)

E.g. news sites, blogs, listings

Today's demo:

- We already collected GitHub repo data via API (4/21, 4/23).
- Now we'll scrape AI news headlines from artificialintelligence-news.com — a site **without an API**.

What Is HTML?

HTML = HyperText Markup Language

- The language that structures every webpage you visit
- Not a programming language — tells the browser how to structure and display content
- For scraping: HTML is the “**raw material**” you parse to extract data

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

HTML Structure

HTML tutorial

[W3Schools](https://www.w3schools.com/html/)

```
<tag attribute="value">content</tag>
```

- **Tags** — Building blocks: <h1>, <p>, <a>, <div>, <table>,
- **Attributes** — Extra info: class="...", id="...", href="..."; help you locate elements
- **Content** — Text between tags, usually what you want to extract
- **Nesting** — Tags inside other tags, similar to nested JSON objects

You don't need to memorize tags. Recognize the pattern.

Your Best Friend: Developer Tools

How to open: Right-click any element → “**Inspect**” (or F12)

- Shows the HTML source with the clicked element highlighted
- See which tags, classes, and IDs contain your target data
- This is the scraping equivalent of reading API documentation

Live demo Inspect artificialintelligence-news.com together

Identify where the article titles, dates and summaries are located.

The Scraping Workflow

1. **Inspect** — Developer Tools — find where data lives in HTML
2. **Fetch** — `requests.get(url)` — download the HTML
3. **Parse** — `BeautifulSoup(html, "html.parser")` — make searchable
4. **Extract** — `.find()`, `.find_all()`, `.select()` — pull out data
5. **Store** — Save to list, DataFrame, CSV

Same logic as the API pipeline, but parsing HTML instead of JSON.

What BeautifulSoup Does

```
from bs4 import BeautifulSoup
html = "<html>...</html>"
soup = BeautifulSoup(html, "html.parser")
```

BeautifulSoup parses HTML and lets you search and navigate it.

Key methods (for reading AI-generated code):

- `soup.find("tag")` — Find the first matching element
- `soup.find_all("tag")` — Find all matching elements (returns a list)
- `soup.select("css.selector")` — Find by CSS selector (class, id)
- `.text` — Remove HTML tags and return clean text
- `.get("href")` — Get an attribute value (e.g., a link's URL)

AI generates the code. You need to understand what each method does.

Demo: Scraping AI News Headlines

Collecting data from a site without API

The Demo Task

Goal: Scrape AI news article data from artificialintelligence-news.com

Why this site?

It has **no public API** — scraping is the primary way to collect this data **programmatically**.

What we'll collect for each article:

- Title — the headline (unstructured text)
- Date — publication date
- Summary — article description / excerpt
- URL — link to the full article
- Category / tags — if available



PHYSICAL AI April 28, 2026
Kakao Mobility details Level 4 autonomous driving roadmap for physical AI



ARTIFICIAL INTELLIGENCE April 28, 2026
Lightelligence's 400% debut is a bet that AI's next bottleneck is the optical interconnect



GOVERNANCE, REGULATION April 27, 2026
Google warns malicious web pages are poisoning AI agents



GOVERNANCE, REGULATION April 24, 2026
Why AI agents need interaction infrastructure

Applying 4D: The Scraping Prompt

Create a Jupyter notebook that scrapes AI news articles from artificialintelligence-news.com.

For each article on the listing page, extract:

- Article title
- Publication date
- Summary / excerpt text
- URL link to full article
- Category or tags (if available)

Handle errors gracefully. Add 2s delay between pages requests. Export to CSV.

Delegation

Discernment

Description

Diligence

Specific: *target site, what to extract, error handling, output format.*

Guided Walkthrough: Scraping Notebook

In VS Code, walk through cell by cell:

Cell 1 Import: `requests`, `BeautifulSoup`, `pandas`, `time`

Cell 2 Fetch the AI News page — inspect raw HTML

Cell 3 Parse with BeautifulSoup — find article elements

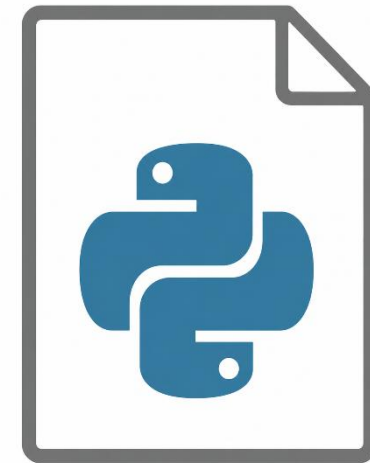
Cell 4 Extract one article: title, date, summary, URL

Cell 5 Define extraction function with error handling

Cell 6 Loop through all articles on the page

Cell 7 Handle pagination (if multiple pages)

Cell 8 Create DataFrame, preview, export to CSV



Week10-2_scraping_ai_news.ipynb

Key Code Patterns

Note: *selectors may need adjustment after inspecting the actual page.*

```
# Fetch the page
response = requests.get("https://artificialintelligence-news.com")
soup = BeautifulSoup(response.text, "html.parser")

# Find all article elements on the page
articles = soup.find_all("article") # or soup.select(".post")

# Extract data from each article
for article in articles:
    title = article.find("h2").text.strip()
    link = article.find("a").get("href")
    summary = article.find("p").text.strip() if article.find("p") else ""
    # Same null-handling as JSON!
```

- If a `<p>` tag exists → extract text and strip whitespace
- If not → return empty string `""`

Hands-on Tasks (~15 min)

Task 1: Run the notebook. Inspect the output CSV — does it have titles, dates, summaries?

Task 2: Modify the code to scrape an additional field (e.g., author name, read time).

Task 3: Try your own target website: ask AI to generate a scraping notebook for a website related to your research project.

Task 3 is the bridge to your final project. Try it!

Common issues: 403 (blocked), dynamic content, different HTML than expected.

Debrief: Scraping vs. API

Scraping is more fragile than APIs:

- HTML structure can change without warning
- Some sites block automated access
- Always add **error handling** and **delays**

For your project: check for an API first. Only use scraping if no public API available.

You now have both tools: API (structured) + scraping (unstructured).

Bonus Time

```
import requests
from bs4 import BeautifulSoup

url = "https://artificialintelligence-news.com"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")

articles = soup.find_all("article")
data = []
for a in articles:
    title = a.find("h2").text.strip()
    link = a.find("a").get("href")
    data.append({"title": title, "link": link})
```



1. What does this script do? (plain English)
2. What data structure is the variable `data`? What does each element contain?
3. What happens if an article has no `<h2>` tag?

What's Next



Assignment 3 due May 10, 23:59

Deliverables:

- Create a GitHub repository
- Submit a short report (3–4 pages, 12pt font, 1.5 line spacing) on Canvas

Refer to the **Final Project Instructions (v3.0)** on Canvas for detailed guidance.

Use AI tools to assist in creating your GitHub repository.

Q&A

dchen@kean.edu CBPM B214

Office hours: Tue & Thu 11:15–12:30 | Wed 14:00–16:30